AIでソフトウェアを開発できても、それが研究の目的に合い、結果を信頼できるものか確かめることが本質的なボトルネックになるという話

SPReADコミュニティ発表の参考文献リスト

発表者　山崎 進

1　ソフトウェアの要求と妥当性確認

発表を視聴した方が，ソフトウェアの要求（requirements）と妥当性確認（validation）を学ぶための文献を，日英両方で読める2点に絞った。ここでは，作る前に「誰が何のために使い，何ができる必要があるか」を明らかにする段階と，できたものが「実際の利用目的を満たすか」を確かめる段階に注目する。

1.1　要求を明らかにする

『ソフトウェア要求 第3版』[Wie14, Wie13]　第1章で要求の基本を押さえ，第7章で利用者から必要なことを聞き出す方法，第11章で確認できる形に書く方法を学べる。第17章は，書いた要求が利用者の目的に合っているかを開発前に見直し，完成後の合否を判断する受け入れ基準につなげる章である。たとえば「測定値を集計したい」という要望なら，その集計で何を判断するのかを聞き，欠測値をどう扱うか，どの件数と平均を出すかを具体的に決めておく。

1.2　できたものが利用目的を満たすか確かめる

『Foundation Level シラバス』[IST23, IST24]　日英とも無料で読める。第1章の1.1節で，書かれた要求を満たすかを調べる検証（verification）と，利用者の必要を満たすかを調べる妥当性確認の違いを押さえ，第2章の2.2.1節にある受け入れテストの説明へ進むとよい。受け入れテストでは，利用者が実際の使い方を想定して，その目的に使えるかを確かめる。たとえば，想定する条件を含む測定データで集計を行い，件数と平均を独立に求めた値と照合したうえで，その集計が研究上の判断に必要な情報を提供するかを確認する。

2　LLM を用いたソフトウェア開発の発達

大規模言語モデル（LLM）を使った開発支援は，コードの続きを提案する機能から，会話を通じて開発を支援する機能へ，さらに，既存のプロジェクトを調べて変更案をまとめる機能へと広がった。ここでは，開発者が AI に何を任せられるようになったかに注目し，主な節目をたどる。

2.1　2020–2021 年：言語モデルからコード補完へ

2020 年の GPT-3 は，指示や少数の例を入力に添えて，さまざまな課題に対応できることを示した [Ope20]。2021 年には，公開コードで追加学習した Codex の論文が，短い説明から Python の関数を生成し，テストで確かめる評価（HumanEval）を報告した [Che21]。同年に技術プレビューが始まった GitHub Copilot は，開発者がエディタで書いているコードやコメントを手がかりに，続きの行や関数を提案する形で LLM を日常の開発作業に組み込んだ [Fri21]。

2.2　2022–2023 年：対話による開発支援

2022 年に公開された ChatGPT は，会話を続けながらコードの問題を調べる例を紹介した [Ope22]。開発者は，候補を採用するか選ぶだけでなく，エラーの理由を尋ね，条件を補足しながら修正を依頼できるようになった。2023 年に発表された GitHub Copilot X は，こうした対話を開発環境の中で使う方向を示した [Git23]。

2.3　2024 年：既存のプロジェクトを対象とする研究

SWE-bench は，GitHub に実際に登録された修正依頼と既存のコードを使い，AI が修正を加えて所定のテストを通せるかを評価する課題集である [Jim24]。同年の SWE-agent は，AI がプロジェクト内のファイルを探し，編集し，コマンドやテストを実行できる仕組みを研究した [Yan24]。評価の対象が，短い関数を生成する能力から，既存のコードを読み解いて修正する能力へ広がったことが分かる。

2.4　2025–2026 年：開発作業を委ね，変更案をレビューする

2025 年に公開プレビューが始まった GitHub Copilot coding agent では，開発者が修正依頼を割り当てると，AI が変更をまとめてプルリクエスト（レビューを求める変更案）を作る。開発者はその変更案にコメントし，追加の修正を依頼できる [Git25]。2026 年には GitHub と VS Code で Claude や Codex を選べる公開プレビューが案内され，複数のエージェントを既存の開発環境から使う形も現れた [Git26]。

2.5　第 2 章の参考文献

まず GitHub と OpenAI の公開記事を読むと，AI に頼める作業がコード補完から対話，変更案の作成へと広がった経緯を追いやすい。研究ではそれをどう評価するかに関心があれば，SWE-bench

の論文に進むとよい。

- **GitHub Copilot の初期公開記事（2021 年）**[Fri21]。コードやコメントから続きを提案する初期の使い方を知るための資料。冒頭の紹介と，エディタ上で提案を受ける例から読める。
- **ChatGPT の公開記事（2022 年）**[Ope22]。コードの不具合を題材にした対話例がある。AI の問いかけに開発者が情報を補い，原因を絞り込む流れに注目するとよい。
- **SWE-bench の論文（2024 年）**[Jim24]。短い関数の生成よりも実際の開発に近い課題をどう作るかを学べる。要旨と冒頭の図から，実際の修正依頼を渡し，変更をテストで評価する流れを追える。
- **GitHub Copilot coding agent の公開記事（2025 年）**[Git25]。修正依頼を AI に割り当て，AI が作った変更案を人がレビューする手順を短い記事で確認できる。2021 年の公開記事と比べると，AI に任せる作業の単位がどう変わったかが分かる。

3　本発表の主張

AI に任せられる実装作業の範囲は広がった　SWE-bench は，既存プロジェクトの修正依頼を AI に渡し，生成した変更を所定のテストで評価する方法を示した [Jim24]。GitHub は，修正依頼を割り当てると AI が変更案を作り，人がレビューできる機能を公開した [Git25]。この二つは，短いコードの生成から，既存コードの変更案を作るところまで AI に委ねられる場面があることを示す。

研究目的への適合性は別に確かめる必要がある　ISTQB のシラバスは，明記された要求を満たすかを確認する検証と，想定する利用環境で利用者の必要を満たすかを確認する妥当性確認を区別している [IST23, IST24]。Sargent はシミュレーションについて，モデルの実装が正しいかと，モデルが意図した用途に対して十分な精度を持つかを区別している [Sar13]。たとえば，指定の集計方法で期待した値が出ることをテストで確認しても，その集計方法で研究の問いに答えられるかは，データの性質と利用目的に照らして判断する必要がある。

結果の信頼性には，計算を再現することと方法の適切さの両方が関わる　全米アカデミーズの報告書は，同じデータとコードなどを用いて整合する計算結果を得ることと，新たなデータを用いた別の研究で整合する結果を得ることを区別する [Nat19]。また，Kapoor と Narayanan は，機械学習を使う研究で，学習と評価の間のデータ漏洩が性能を過大に見せる事例を報告した [Kap23]。同じ手順で数値を再現できても，その数値が研究上の結論を支えるかは，評価手順の前提に左右される。

本発表がボトルネックと考える作業　以上を踏まえ，本発表では，AI に実装を委ねられる場面でも，研究目的への適合性と結果の信頼性を確かめる作業が，研究成果として使える状態に到達するための本質的なボトルネックになると考える。ここでの「ボトルネック」は，実装を速めても，確認の根拠がそろうまで研究上の結論を確定できないという意味である。たとえば Becker らの実験が測ったのは，経験のある開発者が既存のオープンソースプロジェクトで作業を完了するまでの時間である [Bec25]。本発表の主張は，こうした開発時間の測定とは区別し，研究用ソフトウェア

の実装と妥当性確認の所要時間の比較は今後の実証課題と位置づける。

謝辞

本発表は，AI for Science 萌芽的挑戦研究創出事業 (SPReAD)JPMXP1726257598 の助成を受けたものです.

本発表の構想の整理，関連研究の検討および論文原稿の作成・推敲に，OpenAI の GPT-6.1 を対話的な支援ツールとして使用した．研究課題と設計方針の決定，提示した結果の確認，ならびに論文の内容に対する責任は著者にある.

北九州市立大学研究補助者の草野遼氏とともに参考文献のレビューをした.

参考文献

[Bec25] Becker, J., N. Rush, E. Barnes, and D. Rein: Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, 2025, arXiv:2507.09089，https://arxiv.org/abs/2507.09089.

[Che21] Chen, M., et al.: Evaluating Large Language Models Trained on Code, 2021, https://arxiv.org/abs/2107.03374.

[Fri21] Friedman, N.: Introducing GitHub Copilot: your AI pair programmer, The GitHub Blog, 2021, available at https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/.

[Git23] GitHub, : GitHub Copilot X: The AI-powered developer experience, 2023, https://github.blog/news-insights/product-news/github-copilot-x-the-ai-powered-developer-experience/.

[Git25] GitHub, : GitHub Copilot coding agent in public preview, 2025, https://github.blog/changelog/2025-05-19-github-copilot-coding-agent-in-public-preview/.

[Git26] GitHub, : Pick your agent: Use Claude and Codex on Agent HQ, 2026, https://github.blog/news-insights/company-news/pick-your-agent-use-claude-and-codex-on-agent-hq/.

[IST23] ISTQB, : テスト技術者資格制度 Foundation Level シラバス, 2023, JSTQB 翻訳，Version 2023V4.0.J02，https://www.jstqb.jp/dl/JSTQB-SyllabusFoundation_VersionV40.J02.pdf.

[IST24] ISTQB, : ISTQB Certified Tester — Foundation Level Syllabus, 2024, 英語版 v4.0.1（v4.0 の改訂版），https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf.

[Jim24] Jimenez, C. E., J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan: SWE-bench: Can Language Models Resolve Real-world GitHub Issues?, in *International Con-*

ference on Learning Representations 2024, 2024, https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.

[Kap23] Kapoor, S. and A. Narayanan: Leakage and the Reproducibility Crisis in Machine-Learning-Based Science, *Patterns*, Vol. 4, No. 9, 2023, https://doi.org/10.1016/j.patter.2023.100804.

[Nat19] National Academies of Sciences, Engineering, and Medicine, : *Reproducibility and Replicability in Science*, The National Academies Press, 2019, https://doi.org/10.17226/25303.

[Ope20] OpenAI, : Language models are few-shot learners, 2020, https://openai.com/index/language-models-are-few-shot-learners/.

[Ope22] OpenAI, : Introducing ChatGPT, 2022, https://openai.com/index/chatgpt/.

[Sar13] Sargent, R. G.: Verification and validation of simulation models, *Journal of Simulation*, Vol. 7, No. 1, pp. 12–24, 2013, https://doi.org/10.1057/jos.2012.20.

[Wie13] Wiegers, K. and J. Beatty: *Software Requirements*, Microsoft Press, 3rd edition, 2013, https://www.microsoftpressstore.com/store/software-requirements-9780735679627.

[Wie14] Wiegers, K. and J. Beatty: ソフトウェア要求, 日経 BP, 第 3 版, 2014, 渡部 洋子 訳, 宗 雅彦 監訳, https://bookplus.nikkei.com/atcl/catalog/14/P98390/.

[Yan24] Yang, J., et al.: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, in *Advances in Neural Information Processing Systems*, 2024, https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.